

Security Framework Cheat Sheet

Member 4 | csrf.php sanitize.php headers.php logger.php

csrf.php

Protects every form and AJAX request from being forged by attackers

Use These in Your Code

csrfField()	Put inside every HTML form — adds hidden token field automatically
csrfMeta()	Put in page <head> — lets JavaScript read the token for AJAX calls
csrfToken()	Get the current token as a string — use when you need the value directly
verifyCsrf()	Call as FIRST line in every POST handler — blocks request if token is wrong
verifyCsrfAjax()	Same as verifyCsrf() but for AJAX requests that send token in header
verifyCsrfAuto()	Auto detects if request is form or AJAX — picks the right check for you
csrfRotate()	Change the token to a new one — called automatically after each valid submit
csrfGenerate()	Create a brand new token and save it to the session
csrfEnsure()	Make sure a token exists — creates one if missing or expired

Internal Only — Never Call These Directly

_csrfValidateToken()	<i>Does the actual token comparison — used inside verifyCsrf() only</i>
_csrfFail()	<i>Sends 403 error and stops the request — called when token is wrong</i>
_csrfAssertSession()	<i>Checks session is open before doing anything — safety guard</i>

sanitize.php

Cleans all data coming in from users and makes it safe to display

Output — Use on Every echo Before Displaying Data

escape_output(\$value)	Make any variable safe to show in HTML — use on EVERY echo without exception
escape_js(\$value)	Make a value safe to put inside a JavaScript string in your HTML
escape_attr(\$value)	Make a value safe to put inside an HTML tag attribute like value="" or data=""

Input Cleaning — Use on Every \$_POST and \$_GET Value

<code>clean_input(\$value)</code>	Remove dangerous characters, null bytes, and extra whitespace from any text
<code>post_str(\$key)</code>	Read a text value from \$_POST and clean it in one step
<code>get_str(\$key)</code>	Read a text value from \$_GET and clean it in one step
<code>post_int(\$key)</code>	Read a number from \$_POST — returns null if it is not a valid number
<code>get_int(\$key)</code>	Read a number from \$_GET — returns null if it is not a valid number
<code>sanitize_int(\$value)</code>	Convert any value to a safe integer — returns null if conversion fails
<code>sanitize_uint(\$value)</code>	Convert to a positive integer only — returns null if zero or negative
<code>sanitize_amount(\$value)</code>	Check transfer amount is valid — must be at least 100 paise (Rs. 1)

Validators — Check if Data Meets Requirements

<code>validate_username(\$value)</code>	Check username is 3-32 characters with only letters, numbers, _ or -
<code>validate_email(\$value)</code>	Check email is in correct format and not longer than 254 characters
<code>normalize_email(\$value)</code>	Make email lowercase and trimmed so it is stored consistently
<code>validate_password(\$value)</code>	Check password has 8-128 chars with uppercase, lowercase, number, symbol
<code>password_requirements()</code>	Get a readable message explaining password rules — show this to users
<code>validate_file_extension(\$f,\$a)</code>	Check if uploaded file extension is in your allowed list
<code>is_empty_input(\$value)</code>	Returns true if the field is blank after removing whitespace
<code>truncate(\$str,\$length,\$suffix)</code>	Cut a long string to a maximum length safely — works with all languages

Content Sanitizers — For Specific Fields

<code>sanitize_bio(\$value)</code>	Clean biography text — removes HTML tags, limits length to database maximum
<code>sanitize_comment(\$value)</code>	Clean transfer comment — strips tags, keeps to one line, max 500 characters
<code>sanitize_filename(\$value)</code>	Make uploaded filename safe — removes path tricks, bad characters, bad extensions
<code>sanitize_header(\$value)</code>	Remove characters that could break HTTP headers — prevent header injection
<code>sanitize_search(\$value)</code>	Clean a search box input — strips HTML and limits to 100 characters

safe_redirect_url(\$url)	Check redirect URL is safe — blocks attempts to redirect to other websites
get_client_ip()	Get the real IP address of the user safely — handles proxy servers correctly
sanitize_ip(\$ip)	Check an IP address is valid before saving it to the activity logs table

headers.php

Tells the browser exactly what it is and is not allowed to do — first line of every page

Main Functions — Use in Every PHP File

send_security_headers()	Send all security headers at once — call this as the FIRST thing in every PHP page
send_asset_headers(\$type)	Send headers for image or CSS or JS files — allows browser to cache these
send_json_headers()	Send headers for AJAX responses — sets correct type and disables caching
enforce_https()	Redirect user to HTTPS if they visit over HTTP — skips this on localhost
no_cache()	Tell browser never to save this page — use on login, transfer, dashboard pages
cache_for(\$seconds)	Tell browser it can save this page for N seconds — for public pages only
send_error_headers(\$code)	Send correct status code for error pages like 403 or 404

Supporting Functions

get_csp_nonce()	Generate a one-time code for this page load — used in CSP and script tags
build_csp(\$nonce)	Build the Content Security Policy string that blocks XSS attacks
build_asset_csp()	Build a relaxed CSP for pages that serve static files
remove_disclosure_headers()	Remove headers that tell attackers which PHP and Apache version you are using
get_own_origin()	Get your app URL — used to set CORS header so only your app can make requests
debug_headers()	Show all headers that were set — for testing only, REMOVE BEFORE SUBMISSION

logger.php

Records everything that happens in the app — critical for detecting attacks during war game

Logging — Call These After Every Important Action

logActivity(\$event)	Main log function — automatically saves user, IP, page and time to database
logSecurityEvent(\$event,\$detail)	Log a security problem — saves to both database AND log file as backup

Brute Force Detection — Use in Login Handler

isIpBruteForcing()	Returns true if this IP has failed login too many times — block them
countRecentFailedLogins(\$ip)	Count how many times this IP failed to login in the last 5 minutes

Query Functions — Use During War Game to Monitor Attacks

getRecentLogs(\$limit)	Get the most recent N entries from the activity log
getLogsByIp(\$ip,\$limit)	Get all actions from one specific IP address — track an attacker
getSecurityEvents(\$limit)	Get only attack-related entries — CSRF failures, brute force, access denied
getAttackSummaryByIp()	Get a summary of attacks grouped by IP — see who is hitting you hardest
getUserLoginHistory(\$uid,\$limit)	Get all login attempts for one user — check if account was targeted
getLiveActivity(\$minutes)	Get all activity from the last N minutes — live monitoring during war game
renderMonitorDashboard()	Show a full monitoring page in the browser — auto refreshes every 15 seconds

Internal Only — Never Call These Directly

_logToDatabase(...)	<i>Save log entry to activity_logs table in MySQL</i>
_logToFile(...)	<i>Save log entry to activity.log file when database is not available</i>
_rotateLogFile()	<i>Rename log file and start a new one when it gets larger than 5MB</i>
_getLogIp()	<i>Get the validated IP address of the current request for logging</i>

Log Event Constants — Use These With logActivity()

LOG_REGISTER	New user registered	LOG_TRANSFER_INVALID	Transfer attempt with bad data
LOG_LOGIN_SUCCESS	User logged in successfully	LOG_SEARCH	User searched for another user
LOG_LOGIN_FAIL	Wrong password entered	LOG_CSRF_FAIL	CSRF token was wrong — possible attack
LOG_LOGIN_LOCKED	Account locked after too many fails	LOG_ACCESS_DENIED	User tried to access a page they cannot
LOG_LOGOUT	User logged out	LOG_BRUTE_FORCE	Too many failed logins from one IP
LOG_PROFILE_VIEW	User viewed their own profile	LOG_FILE_UPLOAD_FAIL	File upload was rejected
LOG_PROFILE_UPDATE	User updated their profile	LOG_SQLI_PROBE	SQL injection attempt detected
LOG_PROFILE_IMG	User uploaded a new profile image	LOG_XSS_PROBE	XSS injection attempt detected
LOG_PROFILE_OTHER	User viewed someone else profile	LOG_PATH_TRAVERSAL	Path traversal attempt detected
LOG_TRANSFER_OK	Money transfer completed	LOG_SUSPICIOUS	Other suspicious activity detected
LOG_TRANSFER_FAIL	Transfer failed — not enough balance		

Golden Rules — Must Follow in Every File

1	Every echo	<code>echo escape_output(\$variable)</code>
2	Every POST string	<code>post_str("fieldname")</code> or <code>clean_input(\$_POST["x"])</code>
3	Every POST number	<code>post_int("fieldname")</code> or <code>sanitize_uint(\$_POST["x"])</code>
4	Every HTML form	Add inside the form tags
5	Every POST handler	<code>verifyCsrf();</code> must be the very first line after <code>session_start()</code>
6	Every PHP page	<code>require headers.php</code> then call <code>send_security_headers()</code> before any output
7	Every key action	Call <code>logActivity(LOG_*)</code> after login, register, transfer, upload
8	Passwords	NEVER sanitize or clean passwords — pass directly to <code>password_hash()</code>
9	Money transfer	Always use <code>sanitize_amount()</code> and wrap in <code>beginTransaction()</code> / <code>commit()</code>

Summary

File	Functions	Protects Against	Call Order
<code>csrf.php</code>	12 functions	Forged form submissions and AJAX attacks from other websites	2nd — after <code>session.php</code>
<code>sanitize.php</code>	27 functions	XSS, SQL injection, path traversal, bad file uploads, header injection	3rd — clean all input
<code>headers.php</code>	13 functions	Clickjacking, MIME sniffing, SSL stripping, data leaks, browser exploits	1st — very first line
<code>logger.php</code>	15 functions	Blind spots — records all attacks so you can detect and respond fast	Last — after action completes
TOTAL	67 functions	Complete custom security framework — no external libraries needed	

■ Never call functions starting with underscore `_` directly — they are internal only

■ Never sanitize or clean passwords — always pass raw password to `password_hash()`

■ Remove `debug_headers()` and any `var_dump()` calls before final submission

■ Set `APP_DEBUG=false` in `.env` before deploying to the VM for war game